

Introduction

This section documents the implementation and evaluation of the testing strategy for the **MedSupplyDrones** drone delivery system, building upon the plan established in LO2. It assesses the effectiveness of the chosen techniques and demonstrates how they ensure system requirements are met. The testing suite comprises 9 dedicated test classes with 70+ individual test methods across unit, component, integration, and system levels, supported by 29 HTTP test files and 29 JSON test fixtures.

Range of Techniques

The testing strategy employs a comprehensive bottom-up approach utilizing multiple complementary testing methodologies:

Comparison to Planned Testing (LO2)

The LO2 test plan specified 56 high-level tests across eight categories. The implemented test suite exceeds this with 70+ test methods organised into dedicated test classes that directly map to LO2 categories: **FR1: EnvironmentConfigurationTest** (3 planned -> 10 implemented), **FR2: IlpRestServiceFetchingTest** (7 planned -> 12 implemented), **FR3: GeometryCalculationsTest** (10 planned -> 22 implemented), **FR4: DroneFilteringMatchingTest** (8 planned -> 20 implemented), **FR5: FlightPathCalculationTest** (11 planned -> 14 implemented), **FR6: CostCalculationTest** (3 planned -> 7 implemented), **FR7: OrderAssignmentTest** (5 planned -> 9 implemented), **FR8: RestApiEndpointsTest** (6 planned -> 14 implemented), and **NFR: NonFunctionalTest** (3 planned -> 10 implemented). Each test class uses JUnit 5 `@Nested` annotations to group related tests by sub-requirement, providing clear traceability to LO2 specifications.

White-box Testing

White-box testing provides visibility into the internal structure and logic of individual components. Test cases target specific execution paths, boundary conditions, and internal state transitions.

- **GeometryCalculationsTest (FR3):** 22 tests organised into 10 nested classes covering Pythagorean distance calculation (FR3.1), closeness threshold validation at exactly 0.00015° boundary (FR3.2-3.4), point-in-polygon detection for inside/outside/boundary cases (FR3.5-3.7), line-segment intersection for no-fly zone crossing (FR3.8-3.9), and corner case handling (FR3.10). Uses 10^{-12} tolerance constant (EPS) throughout.
- **FlightPathCalculationTest (FR5):** 14 tests validating path invariants including move distance exactly 0.00015° apart (FR5.1), path start/end at service points (FR5.2-3), delivery point proximity (FR5.4-5), hover point detection (FR5.6-7), no-fly zone point avoidance (FR5.8), boundary crossing detection (FR5.9), maxMoves limits (FR5.10), and obstacle routing (FR5.11). Uses helper methods `parseGeoJsonCoordinates()` and `segmentIntersectsPolygon()` for path validation.
- **DroneFilteringMatchingTest (FR4):** 20 tests including `@MockitoExtension` unit tests with mock `DroneQueryService`. Tests filtering by cooling (FR4.1), heating (FR4.2), capacity with boundary values (FR4.3), maxMoves (FR4.4), day-of-week availability (FR4.5), time window availability (FR4.6), empty results for impossible constraints (FR4.7), and drone-service point associations (FR4.8).

Black-box Testing

Black box testing treats components as opaque interfaces, validating that specified inputs produce expected outputs without knowledge of internal implementation details.

- **RestApiEndpointsTest (FR8):** 14 tests across 6 nested classes validating /api/v1/calcDeliveryPathAsGeoJson endpoint behaviour. Tests valid GeoJSON responses (FR8.1), malformed JSON handling returning 400 Bad Request (FR8.2), missing required fields (FR8.3), invalid field values including negative capacity and out-of-range coordinates (FR8.4), empty delivery list handling (FR8.5), and response time under 30 seconds (FR8.6). Uses @SpringBootTest with @AutoConfigureMockMvc for realistic HTTP simulation.
- **HTTP Test Files (29 files):** Parameterised .http files in src/test/resources/api-tests/ organised by endpoint: CalcDeliveryPath (3 files for complex/high/low capacity scenarios), DistanceTo (9 files including boundary and invalid inputs), IsCloseTo (4 files), IsInRegion (4 files including unclosed polygon), NextPosition (7 files for all cardinal directions), and QueryAvailableDrones (2 files). Each references corresponding JSON fixtures in json_test_data/.

Functional Testing

Functional testing validates that the system meets its documented requirements from LO1 and the test specifications from LO2.

- **EnvironmentConfigurationTest (FR1):** 10 tests validating ILP_URL environment variable handling. Tests valid HTTPS/HTTP URLs with paths and ports (FR1.1), missing/null/empty URL error messages (FR1.2), and malformed URL detection including missing protocol, spaces, wrong scheme (FTP), and missing host (FR1.3). Implements IlpUrlValidator inner class mirroring application validation logic.
- **IlpRestServiceFetchingTest (FR2):** 12 tests covering external service error handling. Tests ResourceAccessException for connection refused (FR2.1), HttpServerException for 500 errors, HttpClientErrorException for 404 errors, JSON validation for /drones, /service-points, /restricted-areas, and /drones-for-service-points endpoints (FR2.3-6), and fresh data fetch verification across multiple calls (FR2.7).
- **CostCalculationTest (FR6):** 7 tests validating cost formula ($\text{costPerMove} \times \text{moves}$). Tests basic cost calculation (FR6.1), determinism across repeated requests, multi-drone cost aggregation (FR6.2), independent cost verification against DroneQueryService data (FR6.3), and edge case handling including zero moves.
- **OrderAssignmentTest (FR7):** 9 tests for order-to-drone assignment. Tests single assignment per delivery (FR7.1), capacity requirement satisfaction including cooling capability verification (FR7.2), availability on specified date/time (FR7.3), graceful handling of impossible deliveries returning empty GeoJSON (FR7.4), and maxMoves constraint enforcement (FR7.5).

Structural Testing

Structural testing employs code coverage analysis to ensure comprehensive path execution across the codebase. IntelliJ IDEA's built-in coverage tools measure class, method, line, and branch coverage metrics.

- Coverage targets: 95% class coverage (45/47 classes), 95% method coverage (134+ methods), 92%-line coverage (627+ lines), and 81% branch coverage across the uk.ac.ed.acp.cw2 package.

- Branch testing is approached through modular code design, ensuring different conditional paths are exercised. Java's lack of native branch testing support is mitigated by structuring code for testability.

Combinatorial Testing

Combinatorial testing systematically tests interactions between input parameters to identify edge cases and unexpected behaviour patterns.

- Boundary value analysis for geometry functions tests points exactly on the 0.00015° closeness threshold, polygon boundaries, and coordinate extremes.
- Multiple dispatch scenarios in queryAvailableDrones test combinations of capacity, cooling, heating, maxCost, and scheduling requirements to verify the strictest constraint is always applied.

Scaffolding

Significant scaffolding infrastructure supports efficient test development and maintenance:

- **Mockito Framework:** @MockitoBean annotations isolate units under test from external dependencies. DroneQueryService and DroneAvailabilityService are consistently mocked to enable deterministic testing without live API calls.
- **MockMvc Integration:** @WebMvcTest and @AutoConfigureMockMvc provide lightweight Spring context testing without starting a full application server, enabling fast endpoint validation with proper HTTP request/response handling.
- **JSON Test Fixtures:** 29 JSON files in src/test/resources/json_test_data/ provide reusable test inputs for CalcDeliveryPath, DistanceTo, IsCloseTo, IsInRegion, NextPosition, and QueryAvailableDrones endpoints.
- **Record-based DTOs:** Java record types (LngLat, DroneInfo, MedDispatchRec, etc.) provide immutable test objects with automatic equals/hashCode, simplifying assertion logic.
- **GraphQL Layer Testing:** DroneQueryResolver and DroneMutationResolver are tested via direct method invocation, demonstrating GraphQL's overfetching prevention capabilities.

Evaluation Criteria for Testing Adequacy

Each testing technique employs specific adequacy criteria to determine when testing is sufficient. The following table maps test classes to their evaluation criteria and the LO1 requirements they address:

Test Class	Adequacy Criterion	LO1 Requirement Addressed
GeometryCalculationsTest	All 16 Direction16 values exercised; boundary tests at exactly 0.00015°, just inside, just outside, polygon inside/outside/boundary cases covered	Accuracy – Distance calculations with 10 ¹² tolerance; Closeness checks , Within 0.00015
FlightPathCalculationTest	All 11 LO2 path invariants tested; return-to-origin verified for all paths; no path points inside any restricted area	Flight path – Start/end at same service point; Safety , Avoid no-fly zones entirely
DroneFilteringMatchingTest	Pairwise coverage of constraint combinations (capacity × cooling × heating × maxCost × availability);	Drone filtering – Find suitable drones based on capacity,

	empty results verified for impossible constraints	cooling/heating, cost, availability
RestApiEndpointsTest	All HTTP status codes tested (200, 400, 404); GeoJSON schema compliance verified; response Content-Type validated	REST API – Expose REST API returning JSON with appropriate HTTP status codes
NonFunctionalTest	All requests complete <30 seconds; identical inputs produce identical JSON outputs (5 repetitions); concurrent request handling verified	Performance – Complete within 30-second budget Determinism – Identical outputs for identical inputs
IlpRestServiceFetchingTest	All error conditions (500, 404, connection refused) produce appropriate exceptions; malformed JSON triggers JsonProcessingException	Interoperability – Integrate with external services Reliability – Graceful error handling

These criteria were selected to provide measurable confidence that the software meets its requirements. The boundary value analysis at the 0.00015° threshold directly validates the specification's closeness definition - GeometryCalculationsTest includes tests at exactly the threshold (FR3.3), just inside (FR3.2), and just outside (FR3.4). The pairwise combinatorial approach for drone filtering efficiently covers constraint interactions without exhaustive testing of all 2⁵ combinations. The 30-second performance threshold matches the ILP coursework specification requirement, with NonFunctionalTest asserting completion time using System.currentTimeMillis() instrumentation. The determinism tests execute identical requests 5 times and compare JSON structures using ObjectMapper.readTree() to ensure byte-for-byte reproducibility.

Conclusion

The comprehensive bottom-up testing strategy has successfully validated the MedSupplyDrones system against its primary functional and non-functional requirements. With 95% class coverage, 95% method coverage, and all performance tests completing under 400ms (well within the 30-second budget), the test suite provides high confidence in system reliability. The combination of unit, component, integration, and system-level testing ensures defects are caught early while end-to-end behaviour is validated. The scaffolding infrastructure (Mockito mocks, JSON fixtures, HTTP test files) enables efficient test maintenance and extension. While areas for improvement exist - particularly in mutation testing and external dependency resilience, the current testing foundation provides a solid basis for production deployment and ongoing development of the emergency medication delivery service.